UNIVERSITY of GREENWICH

COMP1549: Advanced Programming

Unit Testing

Dr Markus Wolf

19th March, 2026 - Week 10

Why do we test our code?

UNIVERSITY of GREENWICH

- Is it to:
  - show what brilliant programmers we are by demonstrating that our program doesn't contain any bugs
- or
  - find the bugs that **certainly exist in our code** before anyone else spots them?

- *"A good test is one that is likely to uncover a flaw that was previously unknown."* (Sundsted 1999)

A Basic Problem of Testing

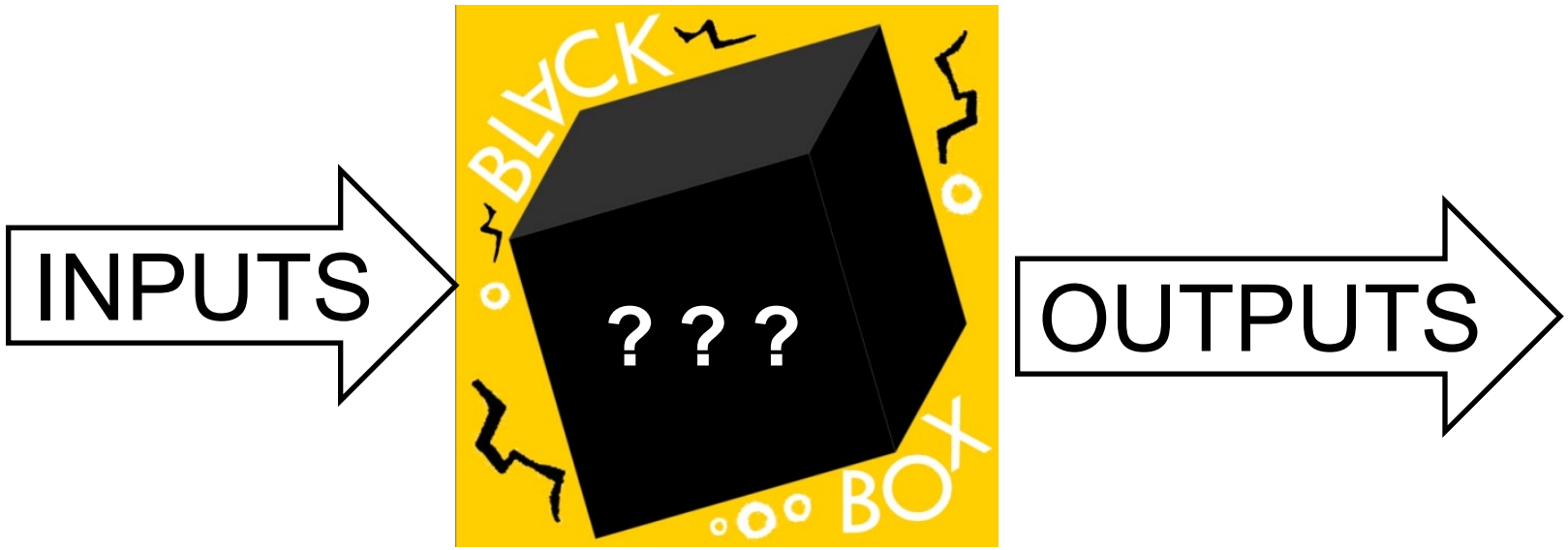
UNIVERSITY of GREENWICH

- Even for simple programs there is **never enough time** to carry out exhaustive testing (e.g. all possible combinations of input)

- Hence the development of **strategies** for creating tests with the **greatest chance** of revealing bugs

- Two key strategies: Black-box testing and White-box testing

Black Box



We don't know what's in the box but we do know what it is meant to do - i.e. given a certain input we can predict the correct output

Choosing Good Test Cases

UNIVERSITY of GREENWICH

- Imagine testing a function called calcCost

Two integer parameters representing:

- price per unit
- quantity of units

If both parameters are >= zero return integer representing cost (price x quantity)

Otherwise throw IllegalArgumentException

Choosing Good Test Cases

UNIVERSITY of GREENWICH

- Choose 8 test cases with what you think are the best chances of uncovering a bug

case	Test case inputs	Expected output
1	price 5, quantity 4	20
2		
3		
4		
5		
6		
7		
8		

Are 8 test cases enough?

White box

UNIVERSITY of GREENWICH

We can see the code in the "box"

Still need to choose test cases with the best chance of catching the bugs

AT THE VERY LEAST choose test cases so at **every line** in the code gets executed at least once during testing

```
public void adjustSpeed() {
        if (speed > 0) {
            if (distance < 10) {
                accelerate = false;
                breakk = true;
            } else if (distance < 50) {
                accelerate = false;
            } else if (speed < speedLimit) {
                accelerate = true;
                breakk = false;
            }
        } else {
            accelerate = true;
            breakk = false;
        }
        resetReadings();
    }
```

What are the minimum number of test cases required to ensure that all the lines of code the method are executed?

Another problem of testing

UNIVERSITY of GREENWICH

- Software doesn't stay tested
- *"But I only made a one line bug fix to the MedicalRecord class to fix that NullPointerException. I can't possibly have caused the whole Hospital system to crash."*
- Oh yes it can!!!!
- Hence the need for regression testing - a suite of **repeatable** tests that can be used to test that things still work after (perhaps seemingly unrelated) modifications have been made

Unit/Component testing

UNIVERSITY of GREENWICH

- The type of testing most likely to be carried out by programmers on their own code
- Shares many techniques and concepts with other types of testing (integration testing, system testing, etc.)
- The rest of the lecture focuses on this

Unit Testing

UNIVERSITY of GREENWICH

- The first unit testing software was called SUnit
  - Developed by Kent Beck (from Extreme Programming fame) and Erich Gamma (one of the Gang of Four, from Design Patterns fame)
  - Used for testing SmallTalk code
- Test frameworks now exist for many programming languages
  - JUnit for Java, CppUnit for C++, NUnit for .NET, many others
  - Known collectively as xUnit
  - Support built into most IDEs

The Problem

- Untested Code may not work
  - Large applications may have many complex code paths that have never been entered during development
  - Worse – code may only work some of the time (because of threading or timing or other external issues – such as network failure)
  - Even worse – some code appears to work, but in actual fact doesn't (this happens far more often than you would believe possible)

The Problem cont.

- Untested code is hard to maintain
  - If you add one piece of functionality – it is possible it will break an existing part of you application. How will you know?
- Untested code is a black box – even with the source code
  - Untested code means the assumptions of the developer are buried deep down in source code. What if their assumptions were wrong?
  - Documentation always falls behind developed code. No one likes doing documentation

Am I Infected?

- Erich Gamma coined the phrase to describe programmers who made writing tests part of their daily programming
- Symptom: seeing a programming problem in terms of tests first and implementation later
- Spend less time debugging and more time designing
- Write many tests – test often – learn testing skills
- The process is called Test-Driven Development (TDD)

Test-Driven Development

UNIVERSITY of GREENWICH

- When you want a write some new code - write the test first
  - Will automatically give you the highest level view of the problem domain you are attempting to address
- Then implement just enough code for your test to pass
  - May well mean developing dummy classes that return set values

Keep Testing

UNIVERSITY of GREENWICH

- Checking your tests fail is as important as checking your tests pass
- When you implement new functionality or change existing code, run all the tests, not just the one you think is affected
  - Ensure that nothing you are introducing in your new code breaks your existing tests

This seems like a lot of work

UNIVERSITY of GREENWICH

- Soon it will save you time
- You know all of your code works
- You add something and immediately know whether your code still works or where it is going wrong
  - The compiler tells you about syntax errors. Your test will warn you of logic error
- You fix something – you know you REALLY fixed it
- You think you fixed it – but you didn't. A well written test will reveal your error straight away

There's more

- A well written test documents your components
  - It shows you how to use your component, not just with words but with **an example**
- If you run your test – your documentation stays up to date (unlike real documentation which is always done last)

Isolation of Tests

UNIVERSITY of GREENWICH

- Tests should be isolated
  - It should be possible to run any test in isolation
  - One test should not be affected by another – the outcome of a test should not be dependent on another test succeeding
  - This enables tests being run in any order
- There is overhead in isolating tests
  - Each test needs to set all state it needs to execute – e.g. open connection to database, instantiate all required objects, etc.

Initialise and Cleanup

UNIVERSITY of GREENWICH

- Creating state for each test can result in very lengthy and repetitive test code
- Code can be simplified by extracting common code to:
  - Create state necessary for a test – setup
  - Return to original state after test – teardown
- Test methods can concentrate on running the actual tests
  - Use the setup and teardown to create state and return to original state, if necessary. May only be required if test makes use of external resources, e.g. DB, server, network connection, etc.

Mock Object

UNIVERSITY of GREENWICH

- A Mock Object is an object designed to ape real behaviour
- When you are developing complex components it can be useful to work to the fixed point of a particular Mock Object until you are ready to implement that object properly

Mock Object

UNIVERSITY of GREENWICH

- Replace expensive, unreliable or slow resources with a mock object
  - E.g. a Database, network connection, sending emails, etc.
- Enables you to perform the test without relying on external sources not to fail
  - Is my code faulty or is it the connection?
- Speeds up the testing
- However – a mock is not the real object, so it's possible your test passes, but the real system fails
  - Make it possible to switch between a mock and the real object

Test Advice

UNIVERSITY of GREENWICH

- What should I test?
  - Every non trivial algorithm
  - Anything that has ever broken
- Tests should be
  - Quick – If they take too long you won't run them so often
  - Self contained - If there is an error in one test it shouldn't propagate to others
  - KISS (Keep It Short and Simple)

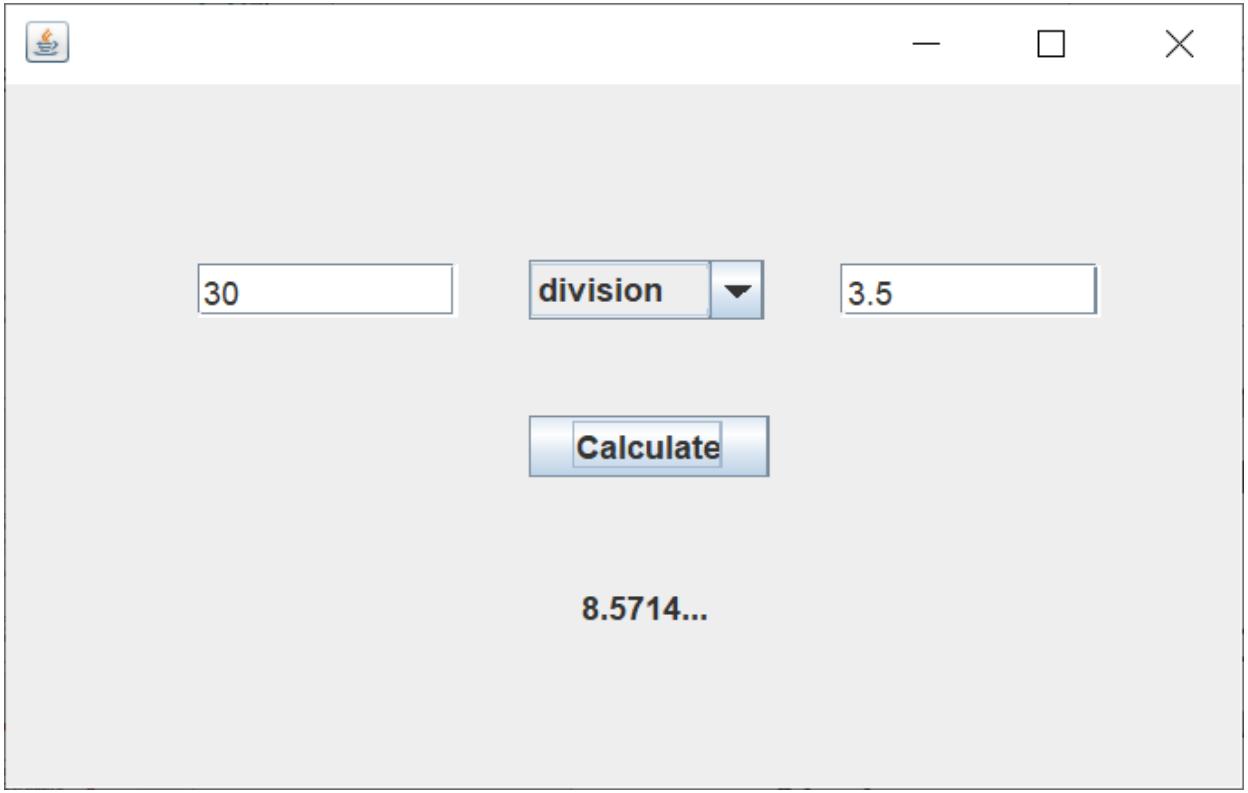
JUnit and Eclipse

UNIVERSITY of GREENWICH

- Support for JUnit tests is built-in to Eclipse
- The testing tools don't offer features for testing UIs (e.g. automatically entering text or clicking buttons)
  - So, very important to put all your application logic in "testable" classes – thin UI layer
- Visibility restrictions apply
  - Every method you want to test has to be visible to the test code – the test code is placed in the same package, so you can test default visibility

Testing Example

- Let's create some unit tests to test the calculator application we created in a previous week (Components)



Testing Example

- We created the application as two separate modules:
  - CalculatorModule – containing the logic
  - CalculatorGUI – containing the user interface layer
- We already have a neat structure where no logic remains in the UI
- The Calculator class contains the methods to carry out the calculations

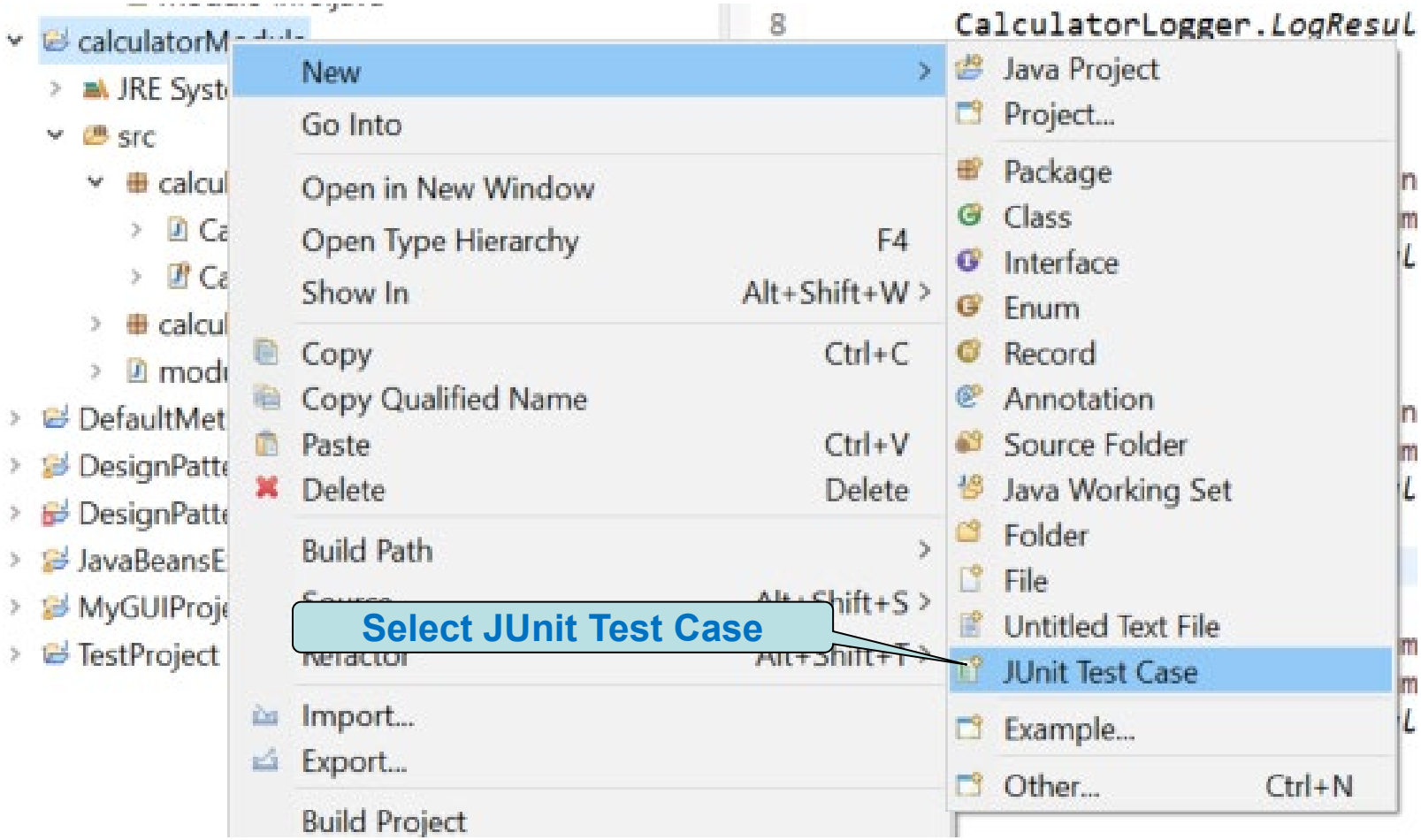
Calculator Class

UNIVERSITY of GREENWICH

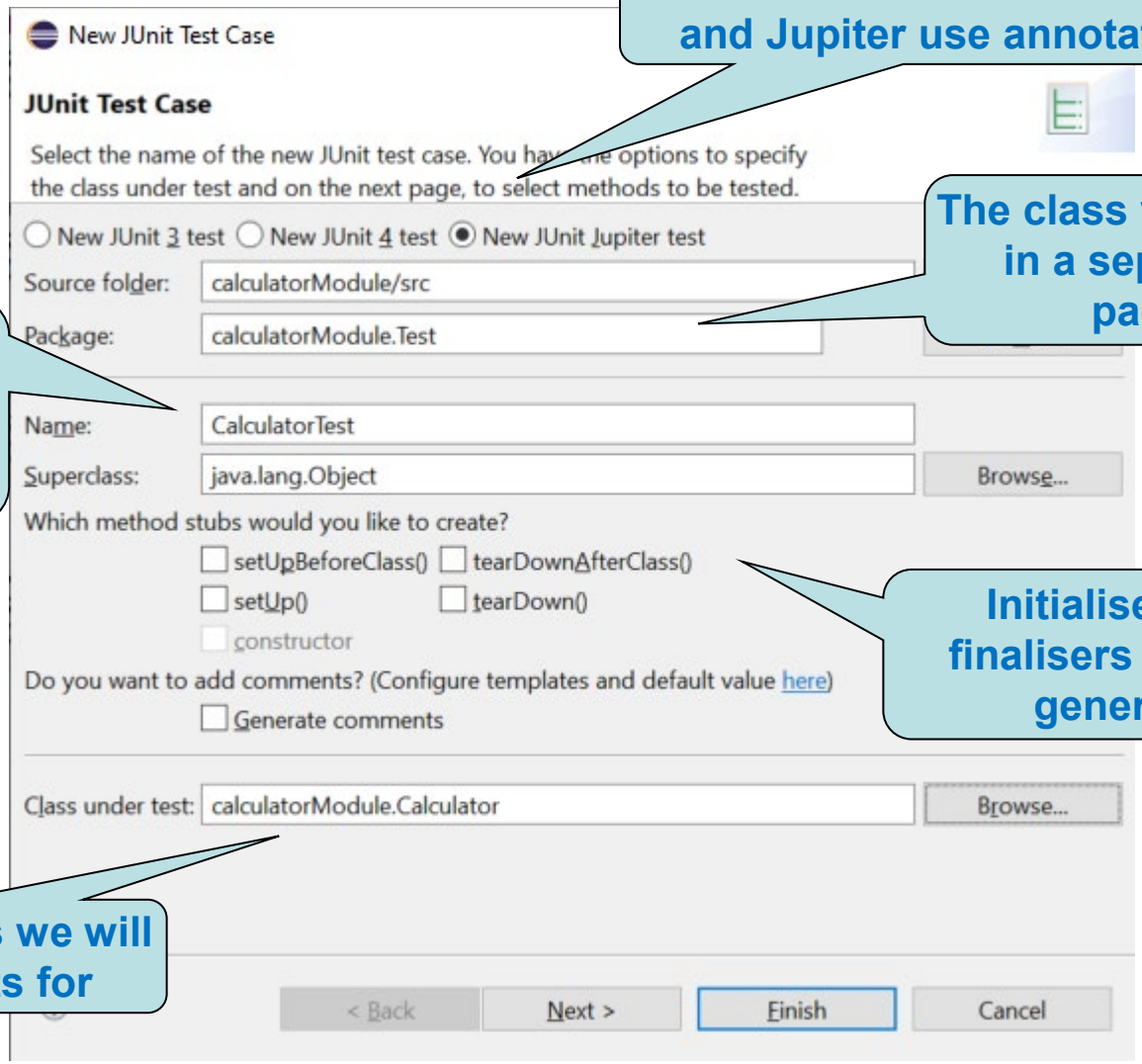
```
package calculatorModule;

import calculatorModule.logger.CalculatorLogger;

public class Calculator {
  public float add(float num1, float num2) {
    float result = num1 + num2;
    CalculatorLogger.LogResult(result,
       CalculatorOperation.addition);
    return result;
  }

  public float subtract(float num1, float num2) {
    float result = num1 - num2;
    CalculatorLogger.LogResult(result,
       CalculatorOperation.subtraction);
    return result;
  }
...
```

Creating a Unit Test

- Right-click on the CalculatorModule project



Creating Unit Tests



Select the test framework Junit 4 and Jupiter use annotations

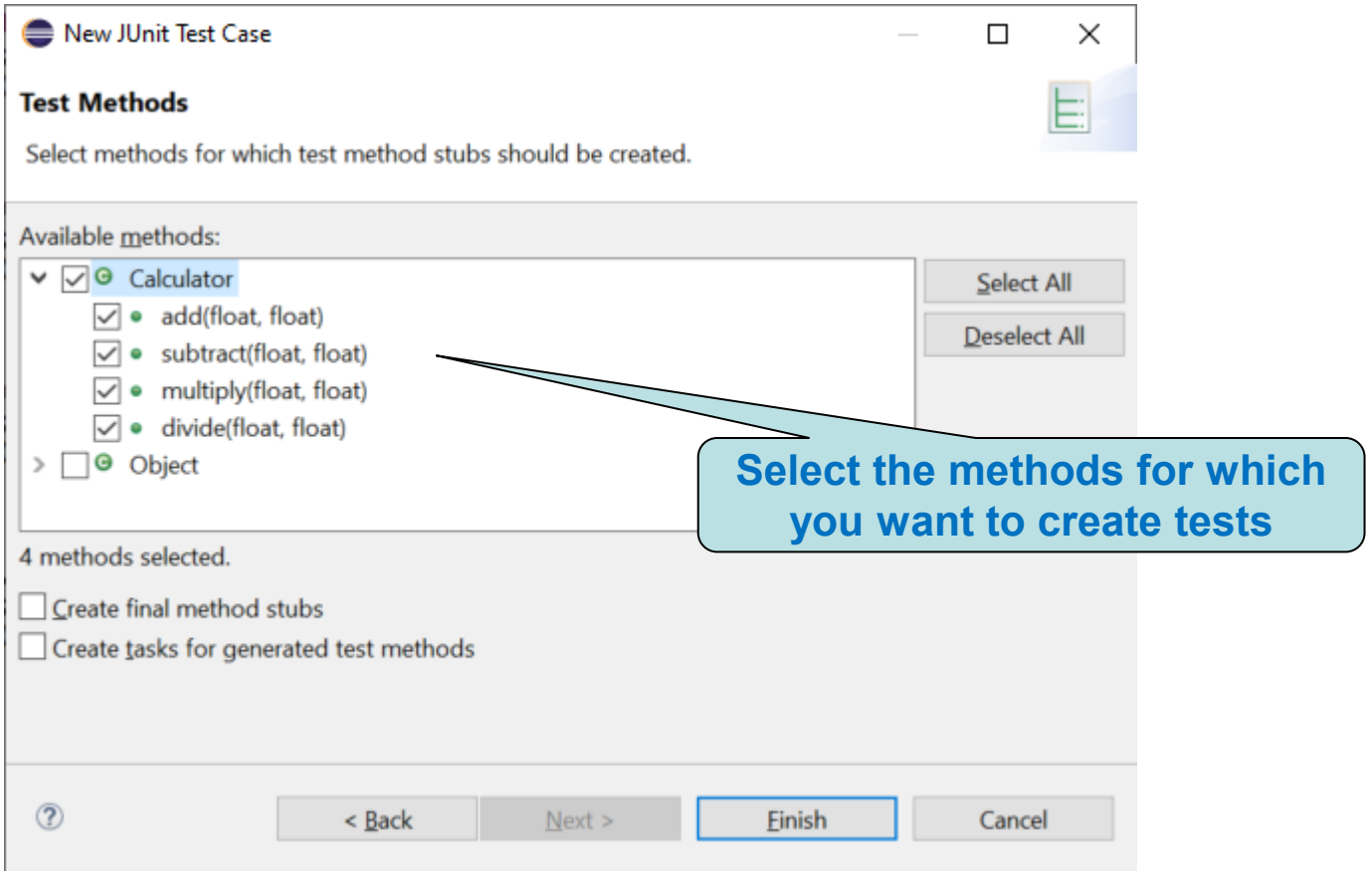
The class will be placed in a separate test package

Set the name for the class containing the unit tests

Initialisers and finalisers could be generated

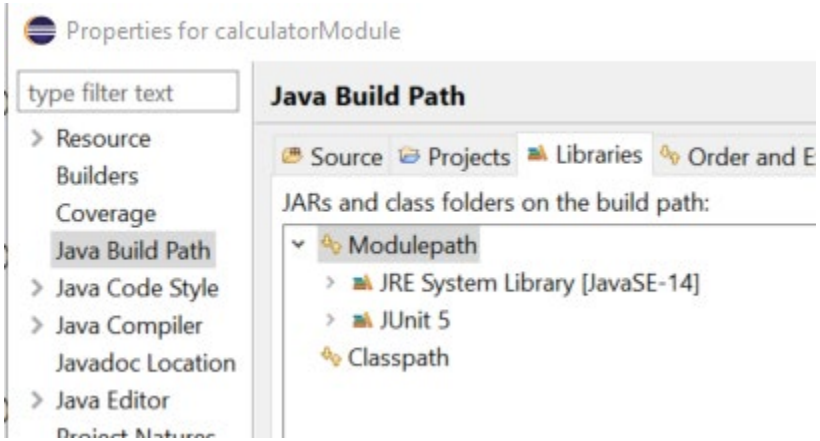
This is the class we will generate tests for

Creating Unit Tests



JUnit and Modules

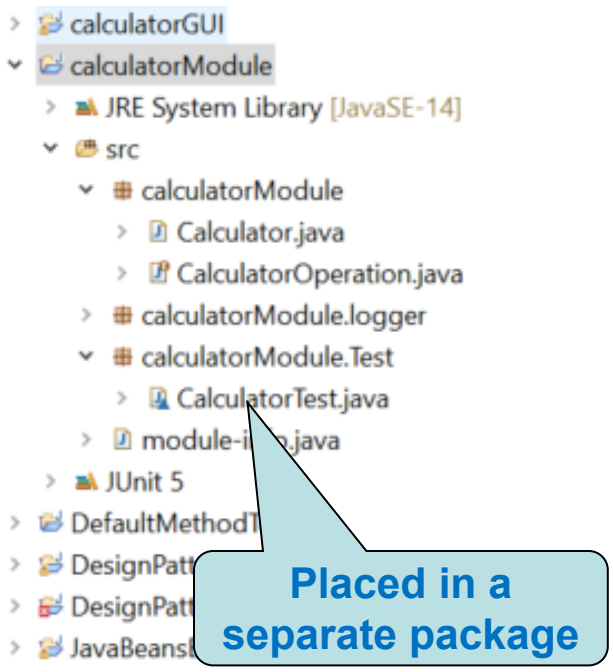
- As our project uses modules, we need to add JUnit 5 to the module path



- We also need to a dependency for *org.junit.jupiter.api* to the module-info file

Test Class

- Eclipse automatically creates a test class and skeleton code




```
1  package calculatorModule.Test;
2
3  import static org.junit.jupiter.api.Assertions.*;
6
7  class CalculatorTe
8
9      @Test
10     void testAd
11         fail("N
12     }
13
14     @Test
15     void testSubtract() {
16         fail("Not yet implemented");
17     }
18
19     @Test
20     void testMultiply() {
21         fail("Not yet implemented");
22     }
23
24     @Test
25     void testDivide() {
26         fail("Not yet implemented");
27     }
28
29 }
30
```

A static import allows us to call static methods directly, without specifying the class name

Attributes

- From version 4 onwards of Junit uses attributes to annotate test elements
  - @Test – defines a test
  - @BeforeAll – runs once before any of the test methods in the class
  - @BeforeEach – runs once before each test in the class
  - @AfterAll – runs once after the last test in the class
  - @AfterEach – runs once after each test in the class
  - @Disabled – makes it possible to ignore a test

Assertions

- Assertions are used to test whether an actual post-condition is the same as an expected post-condition
- JUnit contains many static methods for assertion in the *Assertions* class
- All static methods in the *Assertions* class will evaluate either to true (the test has passed) or false (not passed), with the exception of *fail* which will always fail
- If more than one assertion exist within one test, the test will stop as soon as the first assertion fails

Assertions

- Some of the static methods:
  - assertEquals – true if both arguments have same value (makes call to Equals method for non-primitive data types)
  - assertSame – true if two references point to the same object
  - assertTrue – true if it is passed something that evaluates to true
  - assertArrayEquals - true if two arrays contain the same elements
  - assertNull – true if what is passed in is null
  - More…

Testing Add()

```
@Test
void testAdd() {
   float a = 2.5f;
   float b = 8.2f;
   Calculator calculator = new Calculator();
   float expectedResult = a + b;
   float result = calculator.add(a, b);
   assertEquals(expectedResult, result);
}
```

This is what we expect

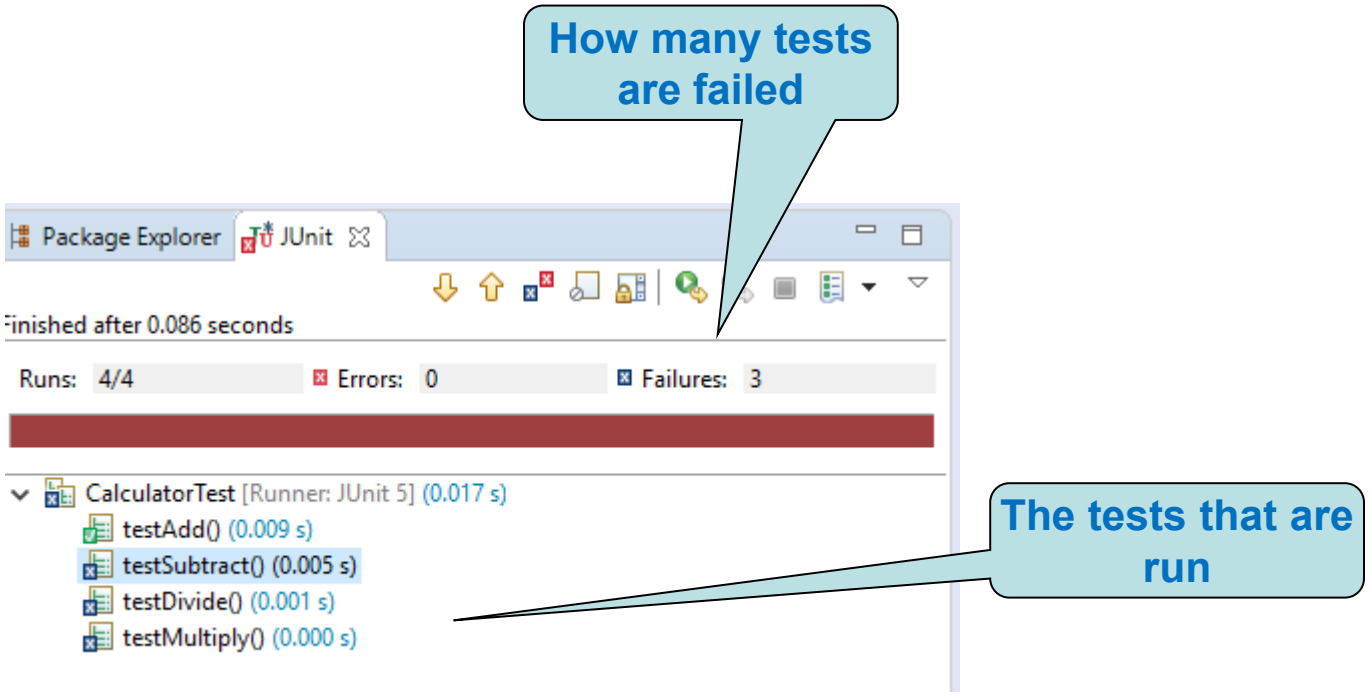
This is what add() returns

Do the results match?

We could use a third parameter - delta (the variation in result that would still pass)

Running Tests in Eclipse

- When you click on the run button, select the Run As JUnit Test option

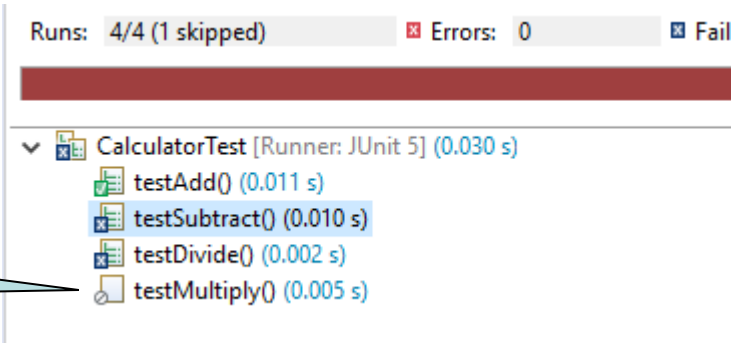


Ignoring Tests

- Sometimes it may be useful to ignore tests
  - E.g. if test functionality is not implemented yet
  - Use *Disabled* annotation

```
@Disabled
@Test
public void testMultiply() {
```

Test will be ignored



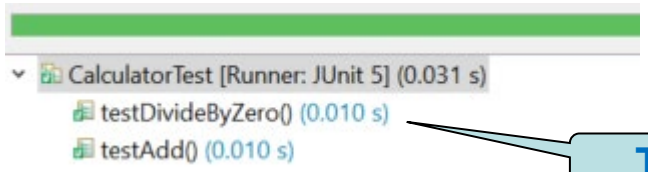
testMultiply is now omitted

Testing Exception

- What if we want to test exception handling?
- It is possible to write tests that only pass if an exception of the expected type is thrown

```
@Test
public void testDivideByZero() {
    int a = 12;
    int b = 0;
    Calculator instance = new Calculator();
    assertThrows(ArithmeticException.class,
        ()-> instance.divideInt(a, b));
```

Tests for an exception



Test passed

Had to add a method that divides integers, as a float would be set to Infinity and not throw an exception

UNIVERSITY of GREENWICH

End of week 10!